Online Real2Sim Calibration of Velocity-Controlled Digital Twins via Reinforcement Learning

Simone Utili*, Andrea Govoni†, Gianluca Palli†

***Abstract*—This paper presents a Real2Sim calibration framework for improving the fidelity of a TIAGo manipulator digital twin in NVIDIA Isaac Sim / Isaac Lab. The method targets joint-space velocity matching by tuning native PhysX joint parameters, namely damping and friction, through reinforcement learning. A curriculum-learning strategy is adopted to progressively calibrate multiple joints, improving optimization stability over direct concurrent training. Experimental results on the TIAGo arm show that the proposed SAC-based approach reduces the velocity mismatch between simulation and measured robot behavior on both sinusoidal and triangular reference signals. The current study is restricted to joint-space velocity behavior and serves as a first step toward richer dynamic digital-twin calibration.**

***Index Terms*—Digital Twin, Real2Sim, Reinforcement Learning, System Identification, Simulator Calibration**

I. Introduction

Robotic manipulators are increasingly deployed in manufacturing, healthcare, and service robotics, where reliability and rapid iteration are essential. In this context, Digital Twins (DTs) provide dynamic virtual replicas of physical assets that support monitoring, simulation, and optimization through continuous data exchange [1]. DT effectiveness strongly depends on simulation accuracy [2], especially when models are used for control design or data-driven learning. However, simulators inevitably introduce modeling simplifications, creating a *reality gap* that degrades both sim-to-real transfer and DT-based decision making [3]. To reduce this gap, Real2Sim pipelines update simulation parameters using real measurements, closing the loop between the physical system and its digital replica [4]. Real2Sim can be effectively addressed via *trajectory matching*, i.e., tuning simulator parameters to minimize the mismatch between real and simulated rollouts [5], [6]. This principle appears in Bayesian/likelihood-free inference [7], learning-based residual correction schemes [8], and Reinforcement Learning (RL)-driven enhancement when gradients are not available [9], [10]. Overall, improving simulator fidelity through Real2Sim is a practical way to support downstream Sim2Real transfer. On the Sim2Real side, many RL pipelines rely on high-level control interfaces (e.g., position or velocity control) [11], whereas torque-level transfer requires more careful identification of friction and damping [5], [6]. Direct RL training on physical hardware is often inefficient, costly, and risky, making simulation a practical alternative when fidelity is sufficient. In this paper, we propose a Real2Sim calibration method that uses RL to tune native PhysX joint parameters responsible for joint-space velocity mismatch. We target joint damping and friction in Isaac Lab and evaluate the approach on the TIAGo manipulator's arm under velocity control, focusing on matching low-level joint velocity responses as a first step toward higher-fidelity digital twins [12], [13]. The paper is structured as follows. Sec. II discusses related work. Sec. III defines the calibration objective. Sec. IV details the training

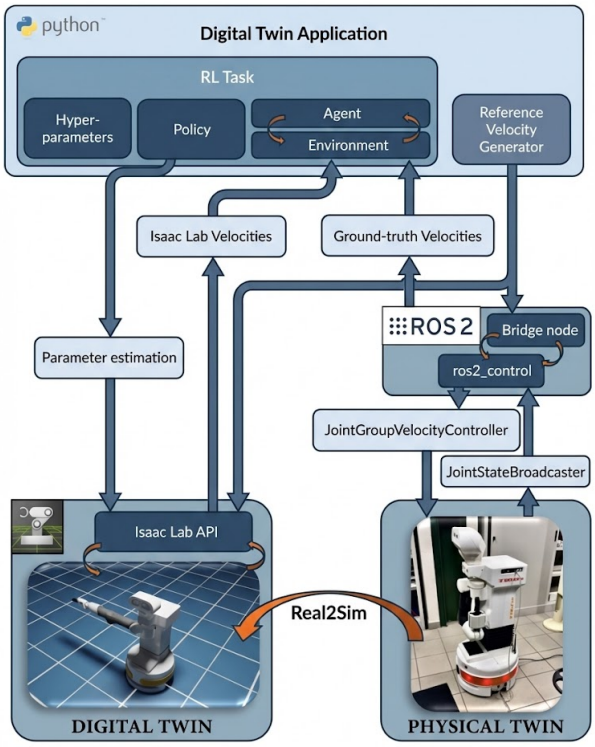


Fig. 1: Real2Sim Digital Twin architecture. A ROS 2 Bridge node enables communication between the standalone Isaac Lab application and the physical robot, where `ros2_control` handles actuation and state feedback.

* A.Govoni and G.Palli Department of Electrical, Electronic, and Information Engineering (DEI), University of Bologna, Italy

*S.Utili is within Delft University of Technology (TU Delft), Delft, The-Netherlands

This work is supported by the Horizon Europe project IntelliMan-AI-Powered Manipulation System for Advanced Robotic Service, Manufacturing and Prosthetics [grant number 101070136].

pipeline. Sec. V reports experimental results, and Sec. VI concludes.

II. Related Works

Recent research has extensively studied simulator calibration and parameter estimation to reduce the sim-to-real gap in robotics. Classical system identification methods [6] rely on analytical models and trajectory-based optimization, but become difficult to apply when simulators involve complex contacts, implicit solvers, or black-box actuator models.

Several data-driven Real2Sim approaches address this limitation by calibrating simulator parameters directly from observed rollouts. SimOpt [4], for instance, iteratively updates simulation parameter distributions from real-world executions using Relative Entropy Policy Search (REPS). Mehta et al. [5] proposed a general framework for evaluating parameter-estimation algorithms and highlighted the importance of incorporating real trajectory data into the calibration loop. Other related works include black-box physics-engine identification [14], one-shot residual correction models such as TuneNet [8], and Grounded Simulation Learning (GSL) [15], which progressively aligns simulated and real dynamics through supervised adaptation.

A related direction formulates simulator calibration through probabilistic inference, e.g., likelihood-free Bayesian approaches [7]. However, when friction is modeled through a small set of independent coefficients, identifiability issues may arise. Bargellini et al. [16] showed on the Franka/Panda platform that static friction estimation can be entangled with other physical parameters, such as inertial terms, motivating calibration strategies that do not rely on a fixed robot-specific friction model.

Vrabič et al. [17] introduce an architecture for bidirectional sim-to-real and real-to-sim experimentation, aligning with the dynamic digital twin concept, but without employing AI for enhancing twin fidelity. While RL has been used to enhance DTs and improve simulator alignment when gradients are unavailable [9], [10], and Sim2Real pipelines often prefer higher-level action spaces for robustness [3], [11], relatively few works have applied RL to achieve accurate, *adaptive* digital twins with real-time parameter estimation capabilities. Notably, existing calibration methods typically assume that a single fixed parameter set suffices, an assumption that breaks down when dissipative effects vary with the operating regime.

Building on these observations, we target a robot-agnostic alternative: *online* calibration of PhysX low-level parameters, namely joint damping and friction, so as to match their joint *velocity response*, and we validate on the TIAGo robot, which features a 7-DOF arm.

III. Velocity-Conditioned Calibration as a System Identification Problem

A. Choice of Calibration Parameters

Modern robot simulators initialize link masses and inertias from CAD models or URDF/SDF descriptions, which remain accurate over the robot's operational life. In contrast, dissipative joint-level phenomena such as friction and viscous damping degrade progressively due to wear, lubrication changes, and mechanical aging, making them the primary source of growing sim-to-real mismatch. A sensitivity analysis of the PhysX joint drive model confirms that these dissipative terms dominate the velocity mismatch in this regime. We therefore restrict calibration to joint damping and friction, keeping all other parameters at their nominal values.

B. Calibration Formulation

As illustrated in Figure 1, the same reference velocity command $\dot{\mathbf{q}}_{\text{ref}}(t)$ is applied to both the real robot and its digital twin, and the discrepancy between the measured velocities $\dot{\mathbf{q}}_{\text{real}}(t) \in \mathbb{R}^n$ and simulated velocities $\dot{\mathbf{q}}_{\text{sim}}(t) \in \mathbb{R}^n$ provides the adaptation signal. In PhysX, joint drives follow the implicit spring-damper law [13], [18]

$$\tau = k_p(q^\star - q) + k_d(\dot{q}^\star - \dot{q}), \tag{1}$$

where k_p and k_d are the position and velocity gains. Since we target velocity matching, we set $k_p = 0$ and calibrate only the dissipative terms.

An adaptation policy π_θ maps an observation $\mathbf{o}(t)$, comprising reference, measured, and simulated velocities, to equivalent simulation parameters

$$\begin{bmatrix} \boldsymbol{\mu}(t) \\ \boldsymbol{\zeta}(t) \end{bmatrix} = \pi_\theta(\mathbf{o}(t)), \tag{2}$$

where $\boldsymbol{\mu}(t), \boldsymbol{\zeta}(t) \in \mathbb{R}^n$ are friction and damping values applied to the n joints. These are not uniquely identifiable physical coefficients but adaptive equivalent parameters that compensate for unmodeled effects. The calibration objective is

$$\boldsymbol{\theta}^\star = \arg\min_{\boldsymbol{\theta}} \sum_{t=1}^{T} \left\| \dot{\mathbf{q}}_{\text{real}}(t) - \dot{\mathbf{q}}_{\text{sim}}(t; \pi_{\boldsymbol{\theta}}(\mathbf{o}(t))) \right\|_1. \tag{3}$$

This observation-conditioned formulation accounts for the fact that dissipative effects depend on motion direction and magnitude and cannot be captured by a single constant parameter over the whole operating range.

IV. Our methodology

We treat the simulator as a black box and use RL to adapt its parameters online from the velocity mismatch between the real robot and its digital twin under the same command input. Because the policy must output two parameters per joint at every control step, namely damping and friction for each of the n active DOFs, conditioned on the instantaneous motion state, the problem is a sequential decision process whose effective dimensionality scales with both the number of joints and the control horizon. Classical offline methods such as Bayesian optimization target one-shot fixed-parameter searches and do not naturally accommodate this time-varying, observation-conditioned mapping; RL handles it natively while requiring no differentiable simulator access. The resulting pipeline is independent of the specific robot model and simulation stack.

A. Curriculum Learning for Multi-Joint Optimization

A straightforward approach to multi-joint calibration is *concurrent* optimization, in which the damping and friction parameters of all n joints are tuned simultaneously. In practice, however, concurrent training proved unstable: the reward signal exhibits high variance due to the distinct dynamics of each joint, and the large search space makes it difficult for the agent to discover effective parameter configurations. As confirmed by the experimental comparison in Section V, these issues lead to slower convergence and inferior final performance.

To overcome these limitations, we adopt a curriculum learning strategy that progressively increases the number of joints being optimized. We assume a fixed joint ordering and, at curriculum stage k, we define an *active joint set* $\mathcal{J}_k \subseteq \{1, \ldots, n\}$ (with cardinality $n_k = |\mathcal{J}_k|$) and a corresponding binary mask $m_k \in \{0,1\}^n$. Only joints in $\mathcal{J}_k$ contribute to the reward and receive parameter updates; inactive joints keep their parameters fixed. The training is structured into sequential stages:

1) *Source task*: The RL agent begins by optimizing damping and friction parameters of a single joint.
2) *Intermediate tasks*: Additional joints are progressively activated, expanding $\mathcal{J}_k$.
3) *Target task*: The agent simultaneously calibrates all joints.

This staged approach reduces complexity in early training and mitigates the risk of converging to poor local optima.

The progression through the curriculum is governed by a signal correlation analysis. Specifically, the *Cosine Similarity* between the simulated and real joint velocity profiles is computed as:

$$C(\dot{q}_{\text{sim}}, \dot{q}_{\text{real}}) = \frac{\sum_{t=1}^{T} \dot{q}_{\text{sim},t} \dot{q}_{\text{real},t}}{\sqrt{\sum_{t=1}^{T} \dot{q}_{\text{sim},t}^2} \sqrt{\sum_{t=1}^{T} \dot{q}_{\text{real},t}^2}}, \tag{4}$$

where T is the number of signal samples. To ensure stable progression, a new joint is introduced when the median cosine similarity across training instances exceeds a predefined threshold ρ.

B. RL Formulation for Calibration

We model calibration as an episodic RL problem, where the agent selects simulation parameter updates and is rewarded for reducing the real-to-sim velocity discrepancy. Training is executed using multiple parallel instances of the simulated environment to improve sample throughput and stabilize optimization.

1) Reward: The reward acts as a loss function, encouraging the agent to minimize velocity errors between simulation and ground truth. At each timestep, it penalizes the Mean Absolute Error (MAE) between joint velocities:

$$R = -\frac{1}{|\mathcal{J}_k|} \sum_{i \in \mathcal{J}_k} |\dot{q}_{i,real} - \dot{q}_{i,sim}| \tag{5}$$

This formulation guides the agent to iteratively refine joint dynamics, increasing fidelity across all degrees of freedom as training progresses.

2) Action Space and Scaling: The agent optimizes joint damping (ζ) and friction (μ) via a continuous action vector with four control parameters per joint:

$$\mathbf{a} = \begin{bmatrix} a_{\mu_\text{scale}} & a_{\zeta_\text{scale}} & a_{\mu} & a_{\zeta} \end{bmatrix} \in [-1,1]^{m \times 4n}, \tag{6}$$

where n is the number of joints, and m is the number of parallel environments.

Given that relevant damping and friction values span several orders of magnitude, a logarithmic scaling strategy is adopted. Raw action values are first scaled to logarithmic exponents:

$$\mu_{\text{scale}} = \frac{a_{\mu_\text{scale}} + 1}{2} \cdot \kappa, \quad \zeta_{\text{scale}} = \frac{a_{\zeta_\text{scale}} + 1}{2} \cdot \kappa \tag{7}$$

where κ sets the explored order-of-magnitude range. These values define the upper bounds of the estimated parameters:

$$\mu_{\max} = 10^{\mu_{\text{scale}}}, \quad \zeta_{\max} = 10^{\zeta_{\text{scale}}} \tag{8}$$

Final parameter values are obtained through linear scaling of the remaining actions:

$$\mu = \frac{a_{\mu} + 1}{2} \cdot \mu_{\max}, \quad \zeta = \frac{a_{\zeta} + 1}{2} \cdot \zeta_{\max} \tag{9}$$

This adaptive rescaling allows the agent to explore both low and high parameter ranges with appropriate resolution.

3) Curriculum Masking: To enforce the curriculum structure, we apply the stage mask m_k to both learning and simulation updates. Concretely, the policy outputs actions for all joints, but parameter updates are applied only on indices in $\mathcal{J}_k$ (inactive joints are masked out). This preserves a constant action dimensionality while ensuring that exploration and credit assignment are restricted to the currently active subset.

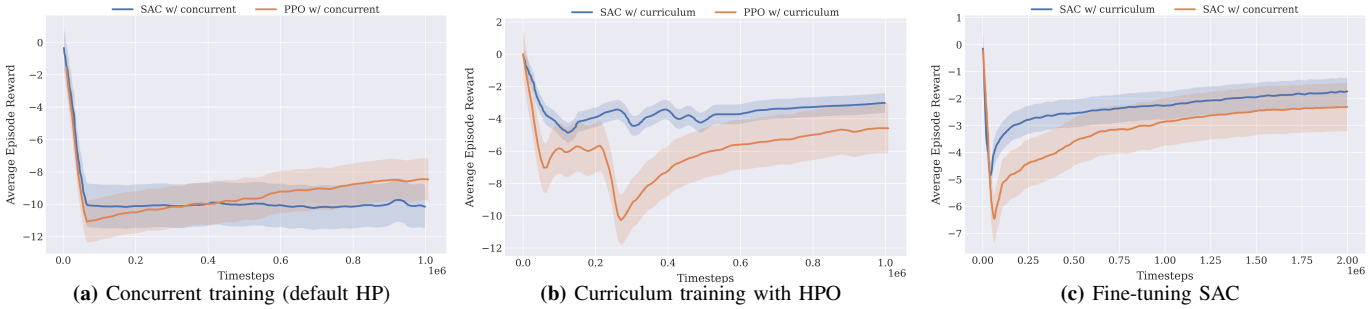
4) Observation Space: The observation vector aggregates reference and measured velocities, simulated velocities, and the curriculum mask m_k:

$$\mathbf{o} = [\dot{q}_{ref}, \dot{q}_{real}, \dot{q}_{sim}, m] \in \mathbb{R}^{m \times 4n}, \tag{10}$$

where:

- $\dot{q}_{ref} \in \mathbb{R}^{m \times n}$: reference joint velocities,
- $\dot{q}_{real} \in \mathbb{R}^{m \times n}$: ground-truth joint velocities,
- $\dot{q}_{sim} \in \mathbb{R}^{m \times n}$: simulated joint velocities,
- $m_k \in \{0,1\}^{m \times n}$: active joint mask.

Inactive joints are zero-padded in the observation vector. The mask m_k conditions the policy on the curriculum stage and ensures that updates are guided only by the currently active DOFs. Note that the policy receives the real joint velocities as input at every step; consequently, the calibration is not a one-shot identification that produces fixed parameters, but an online adaptation scheme that continuously adjusts the simulator dissipative terms based on measured data.



(a) Concurrent training (default HP) **(b)** Curriculum training with HPO **(c)** Fine-tuning SAC

Fig. 2: Training performance comparison (smoothed mean reward $\pm$ std). (a) SAC vs PPO with default hyperparameters under concurrent 7-DOF optimization (10^6 steps, 128 envs). (b) SAC vs PPO with curriculum learning and HPO-tuned hyperparameters (30 trials each, 10^6 steps). (c) Fine-tuning stage (2×10^6 additional steps): best curriculum-trained SAC vs best concurrent SAC, both starting from their respective HPO checkpoints.

C. Training Protocol

We evaluate representative on-policy and off-policy actor-critic methods for continuous actions, and select the best-performing configuration according to the final episode reward. In addition to curriculum progression, we apply a fine-tuning stage on the target task (all joints active) to refine the final parameter estimates. Hyperparameters are tuned through an automated Hyperparameter Optimization (HPO) pipeline over standard RL parameters. At deployment time, the trained policy runs alongside the physical robot: at each control step it receives the current real velocities and updates the simulator parameters accordingly, maintaining a live connection between the digital twin and the physical system.

V. Results

A. Experimental Setup

We evaluate the proposed Real2Sim calibration pipeline on the 7-DOF TIAGo arm under joint-velocity control. The digital twin is implemented in Isaac Lab (PhysX), and the calibrated parameters are the joint damping and friction coefficients. Drive stiffness is set to zero to avoid position regulation effects. The simulation time step is fixed to $\Delta t = 0.01\,\mathrm{s}$, matching the control frequency of the physical robot.

The calibration target is given by joint-velocity measurements collected from the real TIAGo arm. During training, damping and friction are parameterized in logarithmic scale with $\kappa = 3$, allowing exploration over three orders of magnitude. Curriculum progression is triggered when the cosine similarity between simulated and real joint velocities, evaluated over a window of $T = 200$ samples, exceeds $\rho = 0.90$.

B. Training Comparison

To improve robustness and reduce overfitting to a fixed command profile, training uses multi-sine joint-velocity references with randomized amplitudes and frequencies, selected within admissible joint limits to avoid triggering safety stops. The reference signals are designed at sufficiently high frequencies and amplitudes such that the simulator can track them accurately, while the real robot exhibits noticeable velocity lag due to its limited bandwidth, thereby exposing the dissipative mismatch that drives calibration.

We compare Proximal Policy Optimization (PPO) [19] and Soft Actor-Critic (SAC) [20], using the Stable-Baselines3 library [21], in two settings: concurrent optimization of all seven joints, and curriculum-based training. Both methods are trained for 10^6 steps; for the curriculum setting, hyperparameters are tuned over 30 trials per algorithm using the search ranges reported in Table I, and the best models are further fine-tuned for an additional 2×10^6 steps.

The resulting learning curves are shown in Figure 2. SAC consistently outperforms PPO, and curriculum learning improves both convergence speed and final reward. The best result is achieved by SAC with curriculum learning, which reaches a final episode reward of -1.71, compared to -2.33 for SAC trained concurrently. The optimal hyperparameter configurations and corresponding rewards are summarized in Table I. These results indicate that staged optimization is more effective than optimizing all seven DOFs simultaneously.

C. Real2Sim Velocity Matching

The final SAC policy is deployed in Isaac Lab without exploration noise and with all seven joints active simultaneously. The policy is evaluated on unseen sinusoidal and triangular references at 0.2 rad/s amplitude across frequencies from 0.25 Hz to 4 Hz (30 rollouts of 300 timesteps each). Qualitative comparisons at 2 Hz are shown in Figures 3 and 4.

TABLE I: Hyperparameter sweep ranges and best configurations for PPO and SAC under curriculum learning, selected from 30 tuning trials per algorithm. *Abbreviations:* lr = learning rate, bs = batch size, buf = replay buffer size, ent = entropy coefficient, γ = discount factor, τ = target smoothing coefficient.

Param.	**Search Range**		**Best Value**	
	SAC	PPO	SAC	PPO
lr	$[10^{-5}, 10^{-2}]$	$[10^{-5}, 3\times10^{-4}]$	2.99×10^{-4}	1.41×10^{-5}
bs	$[64, 512]$	$[64, 512]$	512	512
$\boldsymbol{\gamma}$	$[0.9, 0.999]$	$[0.9, 0.9999]$	0.905	0.918
ent	$[10^{-3}, 10^{-1}]$	$[0.0, 0.01]$	0.0067	1.07×10^{-4}
$\boldsymbol{\tau}$	$[10^{-3}, 0.1]$	–	0.059	–
buf	$\{50, 100, 500\}$k	–	500k	–
n_steps	–	$[256, 2048]$	–	64
n_ep.	–	$[3, 20]$	–	9
Envs.			128	256
Reward			$\mathbf{-3.02}$	-4.58

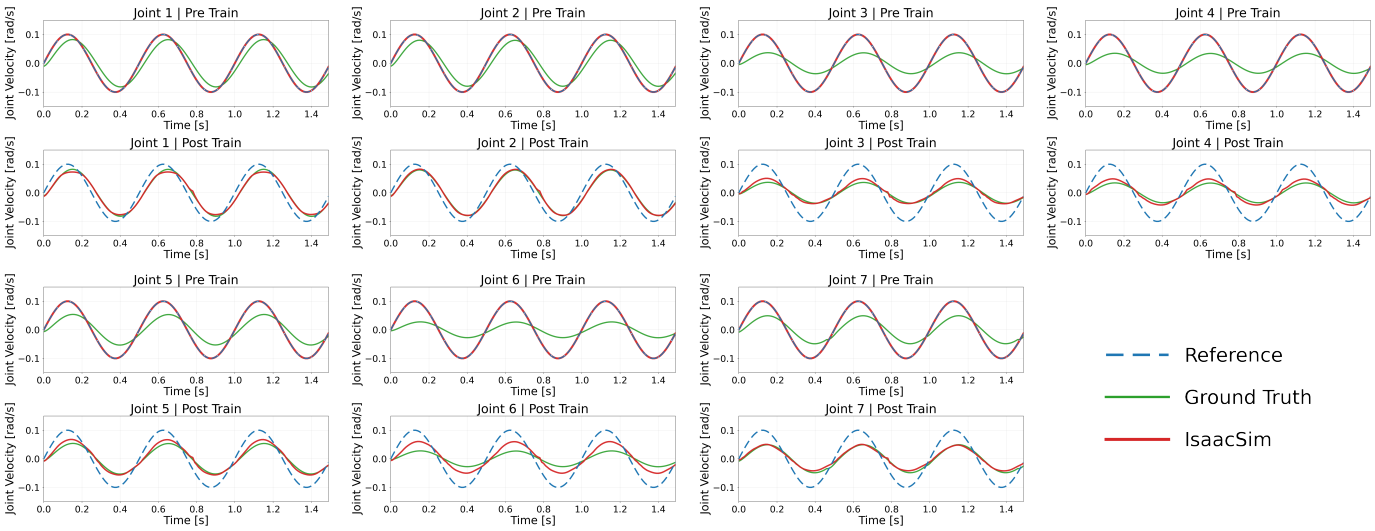


Fig. 3: Sinusoidal joint velocity comparison before (Pre Train) and after (Post Train) Isaac Lab calibration, using the physical TIAGo robot as ground truth.

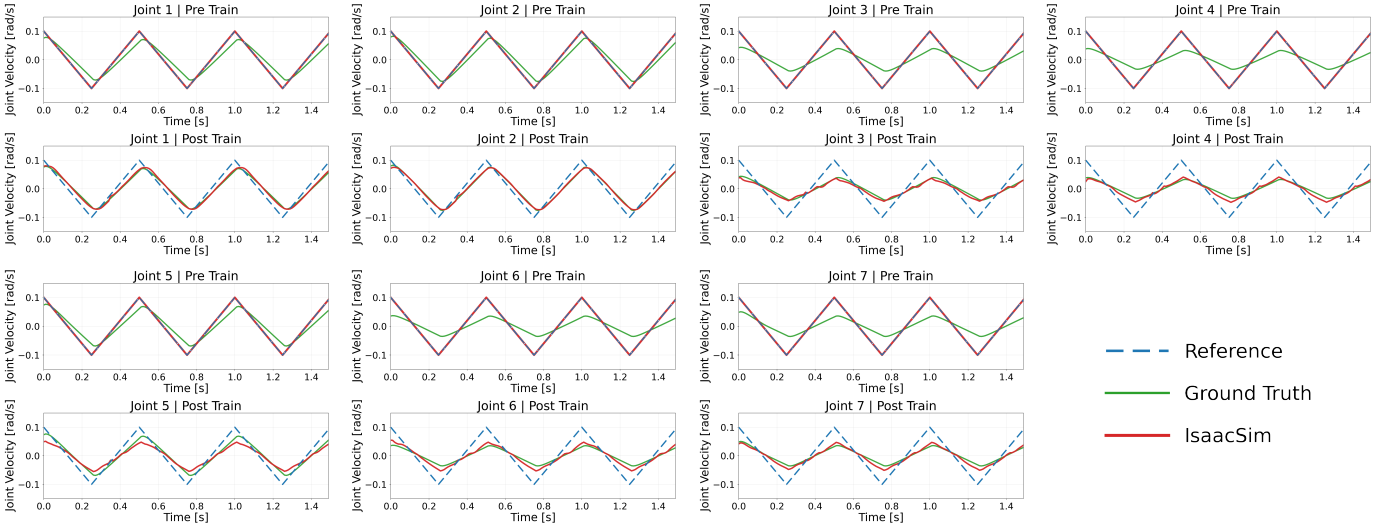


Fig. 4: Triangular joint velocity comparison before (Pre Train) and after (Post Train) Isaac Lab calibration, using the physical TIAGo robot as ground truth.

TABLE II: Mean absolute and standard deviation joint velocity error for sinusoidal commands of amplitude 0.2 rad/s at different frequencies, averaged over seven joints and 30 rollouts.

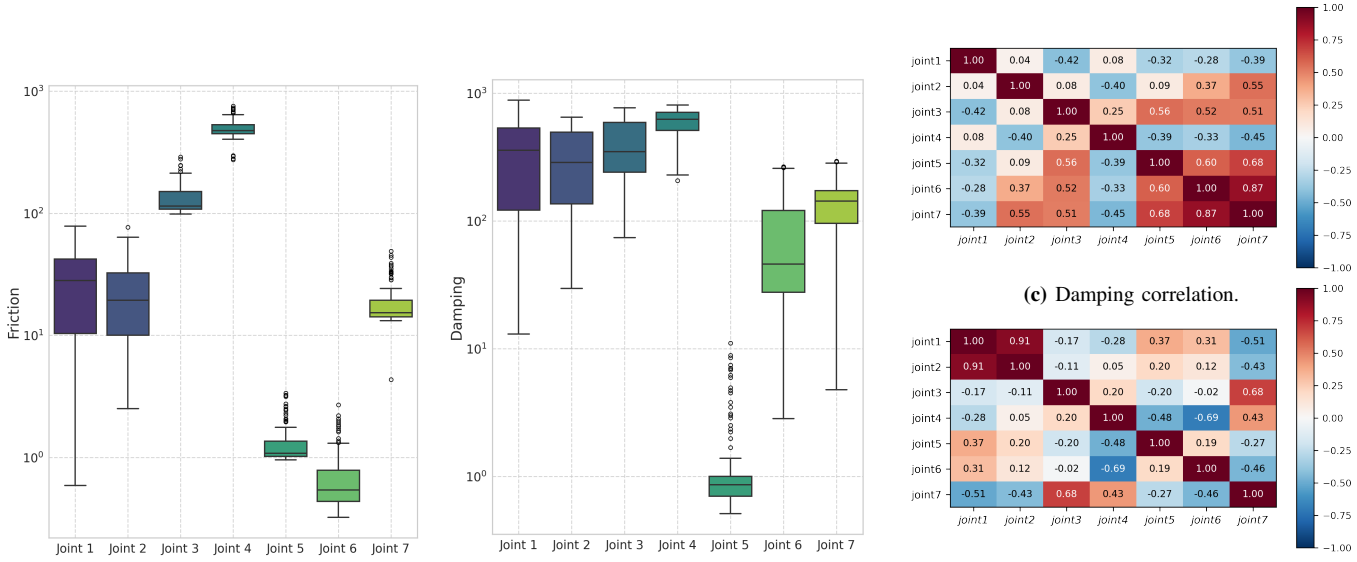
Frequency (Hz)	Uncalibrated Twin		Adapted Twin	
	MAE	Std.	MAE	Std.
0.25	0.004	0.002	0.002	0.001
0.50	0.009	0.003	0.003	0.001
1.00	0.019	0.005	0.004	0.002
2.00	0.036	0.008	0.005	0.002
3.00	0.058	0.012	0.008	0.003
4.00	0.081	0.016	0.011	0.004

Table II reports the mean absolute error averaged over all joints. At low frequencies the pre-train mismatch is already small, but it grows substantially as the real robot lags behind commanded accelerations at higher frequencies. The calibrated twin captures this trend: at 2 Hz the error drops from 0.036 to 0.005 rad/s (sinusoidal) and from 0.026 to 0.005 rad/s (triangular), reductions of approximately 86% and 81%.

VI. Conclusions

This paper presented an RL-based Real2Sim framework to improve digital twin fidelity through online calibration of joint damping and friction in Isaac Lab. Experiments on the TIAGo manipulator showed that curriculum learning, combined with SAC and hyperparameter optimization, consistently outperformed concurrent training and PPO, substantially reducing the sim-to-real velocity mismatch across all seven joints.

These results support RL-driven calibration as a practical solution when simulator dynamics are not analytically accessible, as in PhysX. Although the current study focuses on velocity matching, the proposed approach provides a promising basis for extending digital twin calibration to richer dynamic



(a) Friction distributions. **(b)** Damping distributions. **(c)** Damping correlation. **(d)** Friction correlation.

Fig. 5: Estimated joint parameters after calibration (3 s episode, 300 samples, 2 Hz sinusoidal reference at 0.2 rad/s). (a),(b) Friction and damping distributions (log scale): values do not converge to unique points but show skewed, multimodal profiles due to the non-deterministic PhysX solver and multiple equivalent configurations, indicating that recovered parameters act as adaptive equivalent quantities. (c),(d) Correlation heatmaps: joint dynamics are largely independent, except for a strong damping correlation between joints 1 and 2 ($r = 0.91$) and friction coupling between wrist joints 6 and 7 ($r = 0.87$), suggesting shared dissipative properties in that subchain.

behaviors and more accurate downstream Sim2Real transfer. Future work will extend the proposed Real2Sim calibration from joint-space velocity matching to end-effector trajectory tracking, enabling a more comprehensive representation of robot dynamics.

REFERENCES

[1] R. Rosen, G. Wichert, G. Lo, and K. Bettenhausen, "About the importance of autonomy and digital twins for the future of manufacturing," *IFAC-PapersOnLine*, vol. 48, no. 3, pp. 567–572, 2015.

[2] J. Albardaner, A. S. Miguel, N. García, and M. Dalmau, "Sim-to-real gap in rl: Use case with tiago and isaac sim/gym," in *European Robotics Forum 2024*. Springer Nature Switzerland, 2024, pp. 344–348.

[3] E. Salvato, G. Fenu, E. Medvet, and F. A. Pellegrino, "Crossing the reality gap: A survey on sim-to-real transferability of robot controllers in reinforcement learning," *IEEE Access*, vol. 9, pp. 153 171–153 187, 2021.

[4] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. D. Ratliff, and D. Fox, "Closing the sim-to-real loop: Adapting simulation randomization with real world experience," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 8973–8979.

[5] B. Mehta, A. Handa, D. Fox, and F. Ramos, "A user's guide to calibrating robotics simulators," in *4th Conference on Robot Learning (CoRL)*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2011.08985

[6] L. Ljung, *System Identification: Theory for the User*. Prentice Hall, 1999. [Online]. Available: https://doi.org/10.1007/978-1-4612-1768-8_11

[7] F. Ramos, R. C. Possas, and D. Fox, "Bayessim: Adaptive domain randomization via probabilistic inference for robotics simulators," in *Robotics: Science and Systems (RSS)*, 2019.

[8] A. Allevato, E. Short, M. Pryor, and A. Thomaz, "Tunenet: One-shot residual tuning for system identification and sim-to-real robot task transfer," in *Conference on Robot Learning (CoRL)*, 2020. [Online]. Available: https://proceedings.mlr.press/v155/allevato21a.html

[9] C. Cronrath, A. R. Aderiani, and B. Lennartson, "Enhancing digital twins through reinforcement learning," in *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)*, 2019, pp. 293–298.

[10] A. Sapronov, V. Belavin, K. Arzymatov, M. Karpov, A. Nevolin, and A. Ustyuzhanin, "Tuning hybrid distributed storage system digital twins by reinforcement learning," *Advances in Systems Science and Applications*, vol. 18, no. 4, pp. 1–12, 2018.

[11] A. Shakerimov, T. Alizadeh, and H. A. Varol, "Efficient sim-to-real transfer in reinforcement learning through domain randomization and domain adaptation," *IEEE Access*, vol. 11, pp. 136 809–136 823, 2023.

[12] "Isaac Lab Documentation," https://isaac-sim.github.io/IsaacLab/main/index.html, 2024.

[13] NVIDIA, "Physx articulations," 2022. [Online]. Available: https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/Articulations.html

[14] S. Zhu, A. Kimmel, K. E. Bekris, and A. Boularias, "Fast model identification via physics engines for data-efficient policy search," in *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.

[15] A. Farchy, S. Barrett, P. Macalpine, and P. Stone, "Humanoid robots learning to walk faster: From the real world to simulation and back," in *12th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2013, pp. 39–46. [Online]. Available: https://www.ifaamas.org/Proceedings/aamas2013/docs/p39.pdf

[16] D. Bargellini, A. Govoni, R. Zanella, and G. Palli, "Closing the sim-to-real gap for dynamics-static friction and inertial parameters: A franka robot case study," in *European Robotics Forum 2024*. Springer Nature Switzerland, 2024, pp. 333–337.

[17] R. Vrabič, J. A. Erkoyuncu, P. Butala, and R. Roy, "Digital twins: Understanding the added value of integrated models for through-life engineering services," *Procedia Manufacturing*, vol. 16, pp. 139–146, 2018.

[18] PhysX Team, "Implicit spring joint drives," 2023, technical document, August 14, 2023. [Online]. Available: https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/Joints.html

[19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[20] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," pp. 1861–1870, 2018.

[21] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html